Agent Research

Fletcher Robson, Randen Brown

Ferris State University

Arti 499: AI Capstone

Dr. Molly Cooper

February 11, 2026

Abstract

Artificial intelligence agents are a major step forward compared to traditional software systems, characterized by their ability to perceive environments, reason autonomously, and execute goal-directed actions through ongoing decision-making loops. Unlike conventional scripts or chatbots that operate on fixed instructions, AI agents adapt to changing conditions by integrating large language models (LLMs), memory systems, external tools, and planning mechanisms. This research provides a detailed examination of AI agents spanning foundational concepts, technical architectures, security frameworks, and societal implications. The goal is to explain AI agents in a clear way while also exploring the bigger picture of how they are changing technology and everyday life.

The study begins by establishing core agent components—perception, reasoning, and action—and distinguishes agents from traditional automation through their adaptive learning capabilities and autonomous decision-making. We explore agent lifecycles, internal functioning mechanisms including short-term and long-term memory systems, tool-calling protocols, and both single-agent and multi-agent architectures.

Essential to this investigation is a security analysis addressing vulnerabilities unique to agentic systems. We map common risks—including prompt injection, data leakage, unauthorized tool execution, over-privileged access, and supply-chain vulnerabilities—to specific agent components. Industry security frameworks are evaluated, with particular emphasis on OWASP Top 10 for LLM Applications and NIST AI Risk Management Framework guidelines. Practical security measures are outlined, including access controls, sandboxed execution environments, secrets management, encryption protocols, and rate limiting strategies.

The research examines human-agent collaboration models, emphasizing augmentation over automation, and the importance of human-in-the-loop oversight for decision making. We analyze agents' societal impact, demonstrating their ability to enhance productivity, accessibility, and research acceleration while acknowledging concerns surrounding over-automation, bias, security risks, and technological dependence.

This work connects theoretical understanding with practical implementation, providing both technical information for developers and accessible explanations for broader audiences. By integrating security considerations throughout the agent development lifecycle, this research contributes to the responsible design and deployment of AI agents that balance autonomy with safety, transparency, and human oversight.

Introduction

Sam Altman, CEO of OpenAI, states, “AI will be the greatest technology humanity has ever developed,” but “if this technology goes wrong, it can go quite wrong . . . the bad case . . . is lights out for all of us” (Altman, 2023). This quote exemplifies both the effectiveness and possible doomsday scenario the advancement of artificial intelligence may bring to the future. Agents are an increasingly popular development within the AI community. It is said that “AI agents will become the primary way we interact with computers in the future" (Nadella, 2024). It is essential that society is appropriately educated on the secure use and development of agents. This research covers the foundations and purpose of agents, agent lifecycles, architecture, agent security and risk analysis, industry security frameworks and standards, data security, societal interactions with agents, and the benefits/risks of agent implementation. An AI agent is a system that can perceive its environment and take actions toward achieving a goal with a degree of autonomy. They represent the next stage in the evolution of intelligent systems. Unlike traditional software programs that follow fixed instructions, AI agents are designed to perceive information, make decisions, and take actions to achieve specific goals. They can adapt to changing environments and improve their performance over time. With the integration of large language models memory systems, and external tools, modern AI agents can complete complex tasks that once required direct human involvement. In recent years, the development of AI agents has accelerated due to advancements in machine learning, natural language processing, and cloud computing. These systems are now being used in areas such as customer service, cybersecurity, healthcare, business automation, and personal productivity. Agents are increasingly becoming more important in society. They assist individuals with daily tasks, support businesses in making data-driven decisions, and help organizations automate complex workflows. Their growing presence highlights both the opportunities for increased efficiency and innovation and the need

for responsible development. However, the rise of AI agents introduces new technical, ethical, and security challenges. Since agents can interact with external systems and handle sensitive data, it is essential to understand how they are built, function, and secured. Issues such as data privacy, misuse, bias, and system vulnerabilities must be carefully considered to ensure safe and responsible deployment. By examining these core components of agents, this paper provides a detailed framework for understanding agent education, development, security, and societal implications in an increasingly AI-driven world.

What are A.I. Agents

A.I. Agents are a subclass of artificial intelligence designed to act autonomously on behalf of a user or system. The naming of an agent implies two things: first, that it is an agent on someone's behalf, second, they have the agency to make their own decisions to achieve a goal. Agents have reasoning and decision-making skills, driven by their partnership with LLMs, to then affect the environment around them based on their goals and inputs. Along with this, they have the ability to adapt to different situations based on their prior knowledge and experience. They can act upon this through memory systems and tool calls that allow them to access external data sources and services. Agents' adaptability and ability to act autonomously without user supervision is what separates them from typical software.

Traditional software has explicit rules and only performs tasks it was specifically programmed to handle. When traditional software faces a problem outside of its existing rules, a manual update is needed to handle new scenarios. Agents on the other hand are adaptive and task-oriented, allowing them to encounter a new situation, reason through it, and determine a new course of action without requiring a manual update (Crescendo A.I., 2026). This makes agents well-suited for complex, dynamic environments where every situation cannot be

anticipated. Beyond adaptability, agents also differ from traditional software in how they handle data and make decisions. “AI agents excel at processing large datasets and making decisions based on those insights. Traditional software, on the other hand, is limited to performing operations based on static inputs” (Deremuk, 2025). Handling large data enables agents to make decisions on a multitude of different scenarios. One final distinction between agents and traditional software is in how they approach automation. Traditional software automates through pre-defined instructions, while agents automate tasks through patterns and data predictions (Deremuk, 2025). Agents' ability to act dynamically, handle large data, and automate through patterns are three core pieces that differentiate them from traditional software.

Agents do not only differ from typical software but other types of artificial intelligence as well. Many artificial intelligence systems are narrow in task completion such as simple classification of images, translating text, predictions, and making recommendations. The core difference is their outputs as basic A.I. systems do not affect the environment around them but rather give an output for a human to then change the environment they are accessing. Agents remove the need for human intervention, allowing them to complete multi-step tasks such as writing and executing code. The ability to act independently through a task with multiple steps is what makes agents different from other AI.

As different tasks require different solutions, there are a multitude of varying agents tailored with different capabilities. One basic type of agent is a rule-based agent such as a chatbot. These operate on a set of if-then statements to affect their environment if something happens. Similar to traditional software, they are limited in adaptability and represent the simplest form of agent behavior. Taking a step up in complexity, goal-based and planning agents navigate beyond fixed rules through reasoning toward a defined objective and analyzing different

plans that lead them to success. Learning agents are another type of agent that use prior experience to improve over-time based on previous outputs. The last most complex type of agent is a multi-agent system that uses multiple specialized agents to complete complex tasks that one agent alone could not handle. These different types of agent systems allow for the use of agents in a wide range of tasks, from simple automated responses to complex multi-step problem solving.

Agents are designed to complete tasks ranging from simple outputs to complex situations that require planning, reasoning, and adapting. For example, a simple task that agents are designed for is customer service automation. Removing the need for human intervention allows for simple customer service needs to be met without taking time away from human representatives. Coding agents are an example of an agent that requires more planning and execution compared to a customer service agent. Coding agents can give a code base for a program creating different folders and dependencies that are needed in a large coding project. These tasks are more complex as they must reason for the structure of a code base, manage dependencies throughout the program, and execute tasks in step-by-step sequences. A final, extremely complex task that agents can be built for is the use of autonomous vehicles. Different from coding tasks, autonomous vehicles have immediate real world effects necessitating the input of immediate sensor data retrieval and decision making. Autonomous vehicles receive information from sensors such as cameras and radar simultaneously to navigate traffic, recognize pedestrians, and respond to unpredictable situations such as someone driving through a red light. Ranging from simple customer service to multi-step programming, and finally complex autonomous vehicles, agents differ in design and complexity to achieve the demands of the task at hand.

All the differences between traditional software, artificial intelligence, and agents are driven by the agent's lifecycle and the core building blocks that support them. This lifecycle enables agents to continuously perceive, reason, and adapt in ways that static systems cannot replicate.

How Agents Function

A.I. agents' ability to continuously adapt and solve problems is their key differentiation between classical software. "An agent's decision loop is the heartbeat of its intelligence. It's what keeps it active, responsive, and goal oriented" (Balla, 2025). AI agents function in this manner through lifecycles and decision loops. The life cycle is as follows: taking inputs, reasoning on the provided inputs, decision making/reasoning, acting on the plan, observing the output, and then repeating. The ability to remember and execute tool/API calls are two key pieces of agents that aid each step of the agent lifecycle. Each of these steps are equally important and allow for human-like actions.

Focusing deeper into each section of the agent's lifecycle, the input step will be first. Accepting input can be simplified to "perceiving the current environment" in which the agent is in (Balla, 2025). Agents can perceive their current environment in several different ways. Their inputs can be auditory, visual, textual, environmental, or predictive (IBM, 2025). In each of these scenarios, data is collected. This data can be collected through various sensors, user inputs, API/tool calls, saved data, and system logs (Bryn Mawr College, 2012). Auditory perceptions allow for the interpretation of speech, analysis of environmental noises, and "interact with users through voice-based communication" (IBM, 2025). Visual perception allows agents to analyze visual data such i.e. a picture or video through techniques such as computer vision. Text-based perception supported through natural language processing allows agents to analyze and output

text. Environmental inputs are multimodal, integrating information gathered from multiple sensing mechanisms beyond traditional visual and auditory channels (IBM, 2025). Predictive perception is also unique as it “allows agents to anticipate future events based on observed data” (IBM, 2025). Predictive perception typically is used for analysis or inference, not typical perception (IBM, 2025). Some modes of perception are “cameras, microphones, LiDAR,” radar, and pressure or temperature sensors for environmental sensing (IBM, 2025). These are the foundations for an agent's data gathering and perception.

The next step in the agent lifecycle is decision-making/reasoning. “Agentic reasoning is a component of AI agents that handles decision-making" (IBM, 2025). This step allows agents to complete tasks “autonomously by applying conditional logic or heuristics” (IBM, 2025). AI agents' goals are different depending on which reasoning system they use. Goal-oriented systems may be reward-based, selecting actions that maximize expected outcomes to achieve optimal results (Rens, 2016). Some agentic reasoning systems are driven by conditional logic. Similar to Python if statements, in conditional logic there will be an if statement that and a then statement where if something happens then rule will be executed. The benefits of this are that they are simple, but if an agent comes across a scenario that is not defined, it will fail. The next logic-based method agents can use to reason are heuristics. Some agents have pre-determined goals in which they would use heuristic reasoning to find the actions that will allow them to reach their goal and then “plan these actions before conducting them” (IBM, 2025). Another reasoning system agents use is ReAct which stands for reason + action. In this method, agents will provide their reasoning process in the first step similar to “chain-of thought reasoning in gen A.I. and large language models” (IBM, 2025). It then will output on the reasoning and observe the results (IBM, 2025). Finally, it will generate “new reasoning based on its observations,” and repeat

“until it arrives at an answer or solution” (IBM, 2025). The last agent reasoning method to be explored is multi agent reasoning. This is when multiple agents collaborate and “solve complex problems” (IBM, 2025). In this scenario, the individual agents will “specializes in a certain domain and can apply its own agentic reasoning strategy” (IBM, 2025). Agents use tool and API calls to also aid in reasoning which will be discussed further later. Some challenges that stem from agentic reasoning are “computation complexity, interpretability, and scalability” (IBM, 2025). Similar to most A.I. systems agents can be costly and hard to run. Agent's reasoning can also be hard to understand as they are complex. Finally, some reasoning can be hard to scale to larger projects that may have complex tasks that have multiple different types of inputs or tasks the agents must complete. Each of these goal-oriented logic-based systems make up the foundations of different agents.

After reasoning and planning, the next step in the agent's lifecycle is to act and output. Depending on the previously defined goals, the agents will have different outputs. “The execution of an action then modifies the environment” (Team R, 2025). When acting, the agent can execute tool calls to gain information or produce responses (Bertell, 2025). Agents effect their environment through actuators “which can be physical (robotic arms) or virtual (generating text, or sending commands” (Team R, 2025). After an agent's action the previous understandings and perceptions will be affected or adapted, which restarts the agent's life cycle (Team R, 2025). This exemplifies the last step of an agent's life cycle analyzing the output and adapting.

The last piece of the agent lifecycle is to observe the output and adapt/adjust the previous lifecycle steps. This is a key difference between agents and typical software as they are able to adapt. “AI agents can evolve and optimize processes, while traditional software requires manual updates” (Deremuk, 2025). “AI agents can learn from their environment,” and “automate tasks

based on data predictions" (Deremuk, 2025). Agents have this ability as they have access to memory and reasoning. Memory and reasoning through tool and API calls are the last two foundational pieces of agent success.

Memory allows agents to have context and adapt to previous adaptations of the lifecycle. Memory is "a central active component… mediating between representations and supporting reasoning and prediction" (Peller-Konrad, 2023) There are four different memory systems that agents can use: short-term, long-term, episodic, and semantic. Starting with short term memory, which "enables an AI agent to remember recent inputs for immediate decision-making" (IBM, 2025). Short-term memory in agents is commonly implemented as a rolling context window that temporarily stores recent inputs, with older information overwritten as new data arrives. The strength of this is data not being held for too long, but the information will not be saved longer than one session. Onto long term memory which "allows AI agents to store and recall information across different sessions, making them more personalized and intelligent over time" (IBM, 2025). LTM is created for permanent storage where the data could be held in databases or other structured knowledge repositories. A common technique for LTM is "retrieval augmented generation, where the agent fetches relevant information from a stored knowledge to enhance its responses' (IBM, 2025). The next agent memory system is episodic memory. Episodic memory revers to "the recall of specific personal experiences" (Mayes, 2007). An agent would look back at previous inputs, actions, and outputs to influence the steps of the agent's lifecycle. Past interactions such as conversation history and task logs are two examples of episodic memory. The last memory system agents use is semantic memory. "Semantic memories refer to facts about the world" (Mayes, 2007). This is where agent training data would come into effect.

Semantics consist of general concepts and relationships that commonly occur. An agent may access this memory tool or API calls. Each of these memory types are key to an agent's success.

One last major foundation of how agents work is their ability to enhance their performance and capabilities through tool calls and APIs. Tools allow agents to access memory outside of their training knowledge. There is a plethora of different tools that agents can call that may be predefined. They also can access other software and databases through API calls to software such as LLM's. "For agents, a tool is any external capability they can invoke: a calendar or email API, a web search endpoint, a document or spreadsheet editor, a database or vector store, a code interpreter," etc. (Wang, 2025). Agent tool calls largely interact through API calls to LLMs, and then an external tool is called containing information or allowing to interact with other web systems (Lambert, 2026). Tools An agent uses tools through a "structured request... an orchestrator executes the tool; results are appended to the context; the model continues generating" (Lambert, 2026). Tool calls are typically executed through large language models in which an API must be used, such as Claude, Llama 3, Mystral, and ChatGPT, which all have different tool capabilities (IBM, 2025). Agents first identify a lack of knowledge to complete a task (IBM, 2025). Then a "tool selection mechanism" that finds the required information to complete the task (IBM, 2025). After the tool has been selected the API interface grants the ability to "send structured queries and receive responses," in an comprehensible format for the agent (IBM, 2025). These steps allow for data reading, code execution, and controlling outside systems (Bertell, 2025). Tools are essential to expand the scope and abilites of an agent, but when accessing outside sources but there is always the risk of data leakage. It is necessary to know where data is going when tools or APIs are used. When using an API, third party services or other models may lead to exposure of data. Data and information can be sent to

LLMs or possibly accessed by other web systems. Keeping data secure and protected is essential to setting up a strong foundation for agents as they have steps in the lifecycle. It is integral to understand where data flows and how long it is stored, especially when choosing a memory system.

Overall, AI agents operate through a continuous lifecycle of perceiving inputs, reasoning, goals, acting on decisions, and adapting based on outcomes. Their success is driven by core components including memory systems, reasoning strategies, and the ability to interact with external tools and APIs. Unlike traditional software, agents are capable of adjusting their behavior through contextual awareness and stored experience, allowing them to solve problems in fluid environments. Understanding how these elements function together is essential for designing reliable and secure agent systems, particularly as their capabilities expand and their integration with external data sources increases. This foundation establishes the basis for examining how agents are implemented and applied in real-world scenarios.

How Agents Work

A.I. agents' ability to continuously adapt and solve problems is their key differentiation between classical software. "An agent's decision loop is the heartbeat of its intelligence. It's what keeps it active, responsive, and goal oriented" (Balla, 2025). AI agents function in this manner

through lifecycles and decision loops. "AI agents operate through a structured and repeated decision-making process commonly described as: Input-Reason-Plan-Act-Observe-Repeat" (Balla, 2025). The life cycle is as follows: taking inputs, reasoning on the provided inputs, decision making/reasoning, acting on the plan, observing the output, and then repeating. The ability to remember and execute tool/API calls are two key pieces of agents that aid each step of the agent lifecycle. Each of these steps are equally important and allow for human-like actions. An agent's lifecycle involves several phases, ensuring continuous monitoring, updating, and optimizing. The initial phase involves defining the problem, scope, goals, and potential return on investment for the agent. The data handling stage focuses on gathering, assessing, cleaning, and processing data, preparing it for AI consumption with proper alignment. High-quality data is essential for effective AI agent functioning. Designing the agent's architecture and the model is developed alongside it. This includes integrating ethical and security practices from the outset. The agent's performance is tested and evaluated to ensure it meets the requirements. This often involves simulating conversations and using both rule-based and LLM-based scoring to measure performance and accuracy. Next, the agent is deployed into the production environment and is scaled across channels, ensuring consistent interaction. The agent must be monitored and optimized, so it is key to adjust based on feedback. Finally, as the agent evolves, risks such as model drift and automation failures emerge. So, it is important to continuously monitor the agent's performance and behavior in a production setting. "The lifecycle of Ai agents is not a static process but a continuous loop of improvement, requiring ongoing oversight and adaptation to maintain effectiveness" (Bianchessi, 2025).

The lifecycle begins when the agent receives input. This may come from a user prompt, system trigger, sensor data, or an API response. The input provides the agent with context and

defines the immediate objective. Accepting input can be simplified to "perceiving the current environment" in which the agent is in (Balla, 2025). Agents can perceive their current environment in several different ways. Their inputs can be auditory, visual, textual, environmental, or predictive (IBM, 2025). In each of these scenarios, data is collected. This data can be collected through various sensors, user inputs, API/tool calls, saved data, and system logs (Bryn Mawr College, 2012). Auditory perceptions allow for the interpretation of speech, analysis of environmental noises, and "interact with users through voice-based communication" (IBM, 2025). Visual perception allows agents to analyze visual data such i.e. a picture or video through techniques such as computer vision. Text-based perception supported through natural language processing allows agents to analyze and output text. Environmental inputs are multimodal, integrating information gathered from multiple sensing mechanisms beyond traditional visual and auditory channels (IBM, 2025). Predictive perception is also unique as it "allows agents to anticipate future events based on observed data" (IBM, 2025). Predictive perception typically is used for analysis or inference, not typical perception (IBM, 2025). Some modes of perception are "cameras, microphones, LiDAR," radar, and pressure or temperature sensors for environmental sensing (IBM, 2025). These are the foundations for an agent's data gathering and perception.

The agent processes the input using its underlying model. Large language models use measured reasoning to interpret meaning, evaluate context, and determine possible next steps. During this stage, the agent assesses constraints, relevant information, and potential actions. "Agentic reasoning is a component of AI agents that handles decision-making" (IBM, 2025). This step allows agents to complete tasks "autonomously by applying conditional logic or heuristics" (IBM, 2025). AI agents' goals are different depending on which reasoning system they use. Goal-oriented systems may be reward-based, selecting actions that maximize expected

outcomes to achieve optimal results (Rens, 2016). Some agentic reasoning systems are driven by conditional logic. Similar to Python if statements, in conditional logic there will be an if statement and a then statement where if something happens then a rule will be executed. The benefits of this are that they are simple, but if an agent comes across a scenario that is not defined, it will fail. The next logic-based method agents can use to reason are heuristics. Some agents have pre-determined goals in which they would use heuristic reasoning to find the actions that will allow them to reach their goal and then "plan these actions before conducting them" (IBM, 2025). Another reasoning system agents use is ReAct which stands for reason + action. In this method, agents will provide their reasoning process in the first step similar to "chain-of-thought reasoning in gen A.I. and large language models" (IBM, 2025). It then will output on the reasoning and observe the results (IBM, 2025). Finally, it will generate "new reasoning based on its observations," and repeat "until it arrives at an answer or solution" (IBM, 2025). The last agent reasoning method to be explored is multi agent reasoning. This is when multiple agents collaborate and "solve complex problems" (IBM, 2025). In this scenario, the individual agents will "specialize in a certain domain and can apply its own agentic reasoning strategy" (IBM, 2025). Some challenges that stem from agentic reasoning are "computation complexity, interpretability, and scalability" (IBM, 2025). Similar to most A.I. systems agents can be costly and hard to run. Agent's reasoning can also be hard to understand as they are complex. Finally, some reasoning can be hard to scale to larger projects that may have complex tasks that have multiple different types of inputs or tasks the agents must complete.

Instead of taking a single step, advanced agents create a plan. Planning can involve breaking a task into smaller subtasks, selecting tools, or determining dependencies. "Research in LLM-based planning shows that structured decomposition improves multi-step task performance"

(Yao, 2022). Goal based agents fall into four categories based on their decision-making style. Reactive agents prioritize immediate objectives and react quickly to environmental changes. They use set rules instead of detailed planning. Planning agents focus on long-term goals by assessing potential actions and their effects. They use an environmental model to estimate outcomes of their actions, selecting the most suitable option for their objectives. Hybrid agents merge the benefits of reactive and planning agents. They respond immediately in urgent situations and deliberately when time and resources are allowed for planning. Learning agents improve decision-making by drawing insights from previous experiences. They adapt their actions by refining their strategies or goals based on feedback from their surroundings. Goal-based agents are effective in complex environments. Their adaptability lets them respond to changing conditions by focusing on goals rather than strict rules. With planning abilities, they assess future outcomes and choose actions that align with long-term objectives, ensuring progress toward their goal. While adaptable and capable of planning they face limitations. Their complexity can be high due to the substantial resources required for generating and evaluating plans in environments with many possible actions or unpredictable changes. Specifying goals can be challenging, particularly with vague or conflicting objectives.

The agent executes the selected action depending on the task. This includes but is not limited to generating a response, calling a tool, accessing a database, or sending an API request. Depending on the previously defined goals, the agents will have different outputs. "The execution of an action then modifies the environment" (Team R, 2025). When acting, the agent can execute tool calls to gain information or produce responses (Bertell, 2025). Agents effect their environment through actuators "which can be physical (robotic arms) or virtual (generating text, or sending commands)" (Team R, 2025). After an agent's action the previous

understandings and perceptions will be affected or adapted, which restarts the agent's life cycle (Team R, 2025).

After acting, the agent evaluates the outcome. It checks tool responses, assesses whether the goal is met, and gathers new information from the environment. If the task is incomplete, the cycle begins again. This iterative loop allows agents to refine their responses and adjust strategies dynamically, making them more flexible than static software systems. This is a key difference between agents and typical software as they are able to adapt. "AI agents can evolve and optimize processes, while traditional software requires manual updates" (Deremuk, 2025). "AI agents can learn from their environment," and "automate tasks based on data predictions" (Deremuk, 2025). Agents have this ability as they have access to memory and reasoning. Feedback loops are essential for an AI agent's continuous learning and adaptation. Agents analyze the outcomes of their actions, and this analysis informs future decisions, leading to improved performance over time. This adaptability means that agentic AI systems can learn from interactions, receive feedback, and adjust their decisions based on experience and changing conditions without needing reprogramming. This process allows agents to refine their strategies, correct errors, and optimize their behavior to better achieve their goals. Without feedback loops, AI agents would function like static software. These loops allow error correction, goal alignment, increased reliability, and autonomous adaptation. They play a major role in security and governance including detecting harmful outputs, preventing repeated failures, and monitoring abnormal tool behavior. By continuously evaluating the results of their actions and adjusting their strategy, agents move closer to achieving their goals with higher reliability.

Memory allows agents to have context and adapt to previous adaptations of the lifecycle. Memory is "a central active component… mediating between representations and supporting

reasoning and prediction" (Peller-Konrad, 2023). Memory is a critical component that allows AI agents to retain the utilization of information over time, like human cognition. It enables agents to maintain context, learn from past experiences, and manage information across extended interactions. There are four different memory systems that agents can use: short-term, long-term, episodic, and semantic. Short-term memory "enables an AI agent to remember recent inputs for immediate decision-making" (IBM, 2025). This type of memory is useful in conversational AI, where maintaining context across multiple exchanges is required. Short-term memory in agents is commonly implemented as a rolling context window that temporarily stores recent inputs, with older information overwritten as new data arrives. The strength of this is data not being held for too long, but the information will not be saved longer than one session.

Long-term memory "allows AI agents to store and recall information across different sessions, making them more personalized and intelligent over time" (IBM, 2025). LTM is designed for permanent storage, often implemented using databases or vector embeddings. A common technique for LTM is "retrieval augmented generation, where the agent fetches relevant information from a stored knowledge to enhance its responses" (IBM, 2025). This type of memory is crucial for AI applications that require historical knowledge, such as personalized assistants.The next agent memory system is episodic memory. Episodic memory refers to "the recall of specific personal experiences" (Mayes, 2007). An agent would look back at previous inputs, actions, and outputs to influence the steps of the agent's lifecycle. Past interactions such as conversation history and task logs are two examples of episodic memory. The last memory system agents use is semantic memory. "Semantic memories refer to facts about the world" (Mayes, 2007). This is where agent training data would come into effect. Semantics consist of general concepts and relationships that commonly occur. An agent may access this memory

through tool or API calls. In depth agent memory architecture can be categorized into three forms. Token-level memory involves storing explicit, editable units of information, offering transparency and suitability for tasks requiring clear reasoning. Parametric memory is ideal as it embeds information into the model parameters for implicit generalized knowledge, ideal for tasks needing deeper understanding. Latent memory operates within the model's internal representational space, optimizing performance and scalability. Each of these memory types are key to an agent's success.

One last major foundation of how agents work is their ability to enhance their performance and capabilities through tool calls and APIs. Tools allow agents to access memory outside of their training knowledge. LLMs are the reasoning core and engine in agentic systems, which handle language understanding, planning, and generating actions. For this to function properly, tools are necessary because they let the agent read data, call APIs, run code, or control systems. "For agents, a tool is any external capability they can invoke: a calendar or email API, a web search endpoint, a document or spreadsheet editor, a database or vector store, a code interpreter," etc. (Wang, 2025). Agent tool calls largely interact through API calls to LLMs, and then an external tool is called containing information or allowing to interact with other web systems (Lambert, 2026). An agent uses tools through a "structured request... an orchestrator executes the tool; results are appended to the context; the model continues generating" (Lambert, 2026). Tool calls are typically executed through large language models in which an API must be used, such as Claude, Llama 3, Mystral, and ChatGPT, which all have different tool capabilities (IBM, 2025).

LLM function calling is a key part that transforms LLMs from text generators into active agents capable of planning, deciding, and interacting with their environment. Through function calling

agents can do several things. After receiving datasets, agents can use function calling multiple times to evaluate relevance, optimize results, and create structured data from unstructured sources. Agents can intelligently manage the RAG process, deciding whether to retrieve information, which tools to use, and which databases are most appropriate. Agents first identify a lack of knowledge to complete a task (IBM, 2025). Then a "tool selection mechanism" finds the required information to complete the task (IBM, 2025). After the tool has been selected the API interface grants the ability to "send structured queries and receive responses," in a comprehensible format for the agent (IBM, 2025). These steps allow for data reading, code execution, and controlling outside systems (Bertell, 2025).

Tools are essential to expand the scope and abilities of an agent, but when accessing outside sources there is always the risk of data leakage. It is necessary to know where data is going when tools or APIs are used. When using an API, third party services or other models may lead to exposure of data. Data and information can be sent to LLMs or possibly accessed by other web systems. Keeping data secure and protected is essential to setting up a strong foundation for agents. It is integral to understand where data flows and how long it is stored, especially when choosing a memory system.

AI agent data traverses various destinations depending on its type and purpose, ranging from direct processing within the agent to external storage and tool interactions. User input is the initial data stream that AI agents receive and process to understand requests and goals. This data is crucial for the agent to formulate a plan and execute tasks. User input, such as natural language queries or specific instructions, is directly processed by the AI agent's language models or internal logic to determine the intent and initiate appropriate actions. Key elements from user interactions are stored in the agent's memory, which includes both short-term memory for

immediate context and long-term memory for factual knowledge and details of past conversations. This allows the agent to maintain continuity and provide personalized experiences across interactions. The planning module uses observations from the environment, involving user input and memory, to assemble appropriate plans for the agent. This ensures the agent's actions are aligned with the user's goals.

When an AI agent needs information not available internally or needs to perform an action in the real world, it makes calls to external systems or APIs. APIs facilitate the collection of data from various sources and formats, integrating them into cohesive datasets for the AI agent to process. Many actions an agent performs involve functions or services provided by external APIs. This includes retrieving specific information, performing calculations, or initiating external processes. In multi-agent systems, APIs can enable seamless communication and information exchange between different AI agents or machine learning models, fostering collaborative problem-solving. Logs and memory storage are vital for an AI agent's ability to learn, adapt, and maintain a coherent operational history. Information logs and memory are used in feedback loops to improve the agent's performance. By storing learned information and user feedback, agents can adjust to user preferences and avoid repeating mistakes. Logs provide a trail of the agent's actions, decisions, and interactions. This is crucial for understanding how responses are formulated, identifying errors, and building trust. Through these interconnected data flows, AI agents can effectively process user input, leverage external tools, and learn from their experiences.

Section 4: Building Blocks & Architecture of Agents

LLM-based agent frameworks provide the technical foundation for constructing, orchestrating, and evaluating autonomous agents and multi-agent systems. Rather than relying on simple,

single prompts, these frameworks introduce architectures that combine reasoning engines, tool integrations, memory systems, planning modules, and evaluation mechanisms into coordinated systems. Their purpose is to enable LLM-powered agents to perform complex, real-world tasks that require multi-step reasoning, environmental interaction, and adaptive behavior. Recent discoveries categorize these approaches into single-agent systems that rely on self-reflection and prompt engineering, tool-augmented systems that connect to APIs and external knowledge sources and multi-agent architectures in which specialized agents collaborate or compete to achieve shared objectives.

Modern agent architectures are increasingly described using unified modeling frameworks. A common abstraction includes an LLM "brain," memory buffers (both short-term and persistent), planning components for goal decomposition, action modules for tool invocation, and security layers for guardrails and policy enforcement. The LLM-Agent Unified Modeling Framework (UMF) outlines five essential modules: planning, memory, profile, action, and security (Hassouna et al., 2024). These abstractions support scalable and interpretable agent design while enabling domain adaptability. System instructions remain in a central architectural layer, but within advanced agent systems, this evolves into what is described as Agentic Prompt Engineering (APE). Unlike isolated prompts, APE treats instructions as structured, evolving "playbooks" that define how an agent reasons, plans, and responds across multiple scenarios. Rather than simply responding to input, the agent is guided by an architectural prompt framework that governs task decomposition, collaboration, and execution strategies. This shift transforms prompts from static commands into behavioral control systems that shape long-term autonomy and alignment (Atoms, 2024).

Memory systems in LLM-based agents have advanced significantly through the concept of agentic memory. Agentic memory refers to an LLM's ability to retain and act upon information across sessions, allowing it to simulate coherent, goal-directed behavior over time. This persistence supports continuity and personalization. Architecturally, agentic memory may include vector databases for retrieval, summarization mechanisms for compressing context, and knowledge graph structures that capture evolving user states in interpretable formats. Industrial frameworks such as LangGraph and LlamaIndex incorporate combinations of summarization, retrieval, and graph-based profiling to enable persistent memory. However, research notes that many existing systems separate short-term and long-term memory functions without fully integrating recency-aware updates and conflict resolution

Tool integration and function calling form another critical component of agent architecture. Rather than embedding all knowledge internally, agents rely on structured communication protocols to access external capabilities. The Model Context Protocol (MCP) provides an open standard for integrating tools and contextual data into LLM workflows in a consistent and scalable manner. Through standardized tool invocation, agents can connect to APIs, enterprise systems, and resources while maintaining interoperability. This abstraction layer supports secure, extensible integration across platforms. The environment serves as the operational domain in which an agent acts and receives feedback. In practical deployments, this may include enterprise networks, cloud infrastructure, user interfaces, or multi-agent collaborative ecosystems. Within structured frameworks, the environment is not external, but a defined interface layer that supplies constraints, policy enforcement, authentication requirements, and system state updates. Secure agent design therefore requires integrating environmental

constraints with guardrail mechanisms, ensuring that actions executed through tools remain aligned with system policies and security standards.

Agent structure is a defining element of agentic architecture, shaping how AI systems automate workflows, coordinate reasoning processes, and adapt to dynamic environments. According to IBM, agentic architecture refers to the structural design that enables LLMs to automate agents in completing complex tasks. This architecture provides the infrastructure that supports autonomous decision-making, interoperability with external systems, and adaptive behavior across changing conditions. Overall, the effectiveness of an AI agent depends not only on the intelligence of the model, but also on the structural framework that organizes planning, action, reflection, and coordination. A well-designed agentic architecture supports core characteristics of agency, including goal planning, forethought, self-reactiveness, and self-reflectiveness. These features allow agents to set objectives, anticipate outcomes, monitor results, and adjust strategies over time. Backend tool-calling mechanisms enable agents to retrieve up-to-date data, optimize workflows, and automatically generate subtasks required to achieve complex goals. As these systems operate, they may adapt to user preferences, improve personalization, and offer more context-aware responses.

A single-agent architecture consists of one autonomous entity that perceives its environment, makes decisions, and executes actions independently. It operates under centralized internal reasoning and does not rely on communication with other agents. This structure offers simplicity, predictability, and speed. Since there is no inter-agent negotiation or coordination overhead, single-agent systems are easier to design, debug, monitor, and maintain. They typically require fewer computational and integration of resources, making them cost-effective for narrowly scoped applications. However, single-agent systems have limitations. They may

become bottlenecks when handling high-volume or highly complex tasks. Their rigidity makes them less suitable for workflows requiring domain specialization or distributed expertise. As tasks grow in scale or interdependency, the single-agent model may struggle to efficiently manage multiple objectives simultaneously.

Multi-agent architectures extend these capabilities by distributing responsibilities across several specialized agents. Each agent may focus on a specific domain, such as analysis, retrieval, optimization, or evaluation, while collaborating with others to solve complex problems. Some agents may use natural language processing while others might apply retrieval-augmented generation (RAG). These systems enable flexibility and scalability because agents can adapt their roles as tasks evolve. Multi-agent designs more closely resemble organizational teamwork models, where specialized actors coordinate toward shared goals.

The structural distinction between centralized and decentralized control determines how decisions are coordinated within agent systems. In centralized architectures, a single orchestrator or supervisory agent acts as the system's core authority. This orchestrator allocates tasks, monitors progress, manages global state, and synthesizes results. Known as the Orchestrator Pattern, this approach ensures predictable, debugable behavior with clear accountability. Centralized systems are well-suited for workflows requiring oversight, traceability, and guaranteed consistency. This is because task routing is controlled by one entity; token usage is often more efficient, reducing duplicate effort. However, centralized control introduces trade-offs. The orchestrator becomes a potential single point of failure and may create performance bottlenecks as the number of agents scales. Sequential coordination can increase latency, limit production, and make centralized models less suitable for real-time.

In decentralized architectures, intelligence emerges through peer-to-peer collaboration. Agents communicate directly, make local decisions, and coordinate without relying on a central authority. Each agent maintains its own state and collaborates as needed, creating a distributed decision-making structure. This design offers high resilience and scalability, as the system can continue functioning even if individual agents fail. Decentralized systems often reduce latency for localized decisions and operate effectively in interactive environments. Nonetheless, decentralized models present coordination challenges. Without a central authority, maintaining consistent global behavior and enforcing system-wide priorities can be difficult. Duplicate work may reduce efficiency, and achieving coherent output becomes increasingly complex as the number of agents grows. These systems are less suitable for workflows requiring strict synchronization or detailed audit trails.

Orchestrators and planners serve as structural coordination mechanisms within agent systems. An orchestrator manages agent interaction, task routing, monitoring, and synthesis. It maintains global visibility and enforces workflow logic. In hierarchical designs, the orchestrator ensures that agents adhere to predefined roles and reporting structures. In hybrid environments, leadership may shift dynamically based on contextual needs, creating collaborative leadership models where agents share responsibility while maintaining structured oversight. Planning modules complement orchestration by decomposing high-level goals into actionable subtasks. Planning incorporates elements of intentionality and forethought, enabling agents to anticipate dependencies and sequence operations logically. Effective planners also support self-reflection and monitoring, allowing agents to evaluate intermediate results and adjust strategy. In advanced systems, planners may generate new tasks autonomously based on evolving objectives, further enhancing adaptive autonomy.

Together, orchestrators and planners form the backbone of structured agent coordination. Both are essential for supporting scalable, domain-extensible AI systems capable of managing complex, multi-step operations.

As AI agents become more capable, their effectiveness increasingly depends on how they communicate with external systems. LLMs alone generate text based on learned patterns, but when combined with APIs, structured function calls, and execution pipelines, they can interact with real-world environments. Understanding how these components communicate is essential to building reliable, scalable, and secure agent-based systems. Application Programming Interfaces (APIs) allow software systems to communicate with one another. In the context of AI agents, APIs enable the agent to access external services such as databases, web search engines, financial systems, cloud platforms, or enterprise software. When an agent needs updated information or must perform an action outside its internal capabilities, it sends a structured request to an API endpoint. The API processes the request and returns a response, typically in a structured format such as JSON.

Function calling is a structured method that allows LLMs to request specific tools programmatically rather than generating free-form text instructions. Instead of saying “Call the weather API,” the model produces a formatted function request with predefined parameters. The system then executes the function and returns structured results. Function calling improves reliability by reducing ambiguity. Since functions are predefined with expected inputs and outputs, the agent interaction with tools becomes more predictable and machine-readable. This is particularly important in enterprise environments where precision and structured communication are required. Function calling effectively transforms LLM output into executable commands while maintaining guardrails that prevent unauthorized or malformed requests.

Tool execution pipelines refer to the structured process through which agent actions are executed, validated, and returned to the system. This pipeline typically includes several stages; decision phase: the agent determines whether a tool is needed, invocation phase: the agent generates a structured function or API request, execution phase: the external tool processes the request, validation phase: the system verifies output integrity and security compliance, integration phase: the result is returned to the LLM for interpretation and response generation. Execution pipelines may include error handling, logging, access checks, and monitoring layers. This layered structure ensures controlled and traceable interactions between the agent and its tools. Tool pipelines are especially important when agents perform actions that affect real systems, such as updating records, executing transactions, or modifying configurations. By structuring execution steps, organizations reduce the risk of unintended consequences while preserving operational transparency.

The agent to tool to environment to agent cycle interprets a goal and selects a tool. The tool interacts with the external environment, whether that environment is a database, a cloud platform, a marketplace, or a physical system. The environment produces a result, which is returned to the agent. The agent then observes the outcome, evaluates progress toward the goal, and determines whether further action is needed. This loop enables iterative refinement. Each cycle introduces feedback that informs the next decision. The loop continues until the goal is achieved, or termination conditions are met. These communication loops are central to autonomous agent behavior. Rather than generating a single output, the agent actively engages with its environment, gathering data, and adjusting strategy. However, this capability also introduces security and governance considerations, as unchecked loops could produce

unintended results. Proper monitoring and permission controls are therefore essential components of safe agent–tool–environment interaction.

Section 6: Agent Security & Risk Analysis

As AI agents become more autonomous and integrated into enterprise systems, security must be embedded across every layer of the architecture. Unlike traditional software applications that execute predefined instructions, AI agents generate decisions dynamically and interact with tools, APIs, and data stores in real time. This creates new risk surfaces that require careful governance and protection. Key components that must be secured include prompts, tool access, APIs, memory stores, user inputs, and outputs. Each of these elements represents both a functional necessity and a potential vulnerability.

Prompts form the foundational instructions that guide an agent's behavior. System prompts define the agent's identity, constraints, rules, and objectives. Since LLM-based agents rely heavily on prompt conditioning, tampering prompts can significantly alter behavior. Malicious prompt injection attacks attempt to override system instructions by inserting adversarial text into user inputs or external content. If not properly handled, the agent may follow unintended or unsafe instructions. To mitigate this risk, system prompts should be isolated from user inputs and not directly modifiable by external actors. Input filtering, role separation (system vs. user messages), and strict policy enforcement help reduce injection vulnerabilities. Additionally, organizations can implement prompt validation layers that evaluate risky phrases or conflicting instructions before passing them to the model. From a governance perspective, prompts should be version-controlled and regularly audited to ensure compliance with policy and ethical guidelines.

Tools give AI agents the power to act beyond text generation. These tools may include database queries, cloud operations, file system access, financial transaction systems, or enterprise software integrations. Since tools enable real-world impact, improper access controls could allow agents to perform unauthorized or harmful actions. Securing tool access requires implementing the principle of lease privilege. Agents should only have access to the minimum tools necessary for their intended function. Role-based access control can define specific permissions based on task type or user authorization level. Additionally, tool calls should be validated through the structured function schemas to prevent malformed or unauthorized execution. Monitoring and logging of tool interactions further enhances accountability and enables incident to respond if misuse occurs.

APIs server as bridges agent and external services, making API endpoints a high-value target for attackers. If credentials such as API keys or tokens are exposed, malicious actors could misuse them for unauthorize date access or service manipulation. Agents frequently operate across distributed systems, so credential protection becomes critical. Credentials should never be hard coded within prompts or model instructions. Instead, securs storage solutions such as encrypted vaults or environment variables should manage secret keys. API calls should be authenticated and encrypted using secure protocols. Rotating keys regularly and limiting API rate limits can reduce the potential damage to compromised credentials. Furthermore, strict auditing of API access logs helps detect abnormal usage patterns early.

Agent memory systems for both short-term conversational memory and long-term persistent storage introduce privacy and integrity challenges. Memory stores may retain user queries, preferences, historical decisions, or enterprise data. If improperly secured, these repositories could leak sensitive information or be manipulated to influence future agent behavior. Memory

systems should implement encryption at rest and in transit to protect stored data. Access controls must ensure that only authorized processes can read or modify memory entries. Additionally, retention policies should define how long data is stored and under what conditions it is deleted, aligning with data protection regulations. Validation checks should also prevent corrupted or maliciously inserted data from influencing future reasoning cycles.

User inputs represent one of the most common attack vectors in AI systems. Adversarial prompts, malicious file uploads, and embedded instructions in external content can compromise the agent's intended operation. Since AI agents often interpret text contextually rather than deterministically, they can be susceptible to manipulation through cleverly crafted inputs. Input sanitation mechanisms should filter potentially harmful content and enforce data-type constraints where applicable. Separation of instructions from data is critical to prevent unintended instruction execution. In enterprise systems, user authentication and session controls also help ensure that only authorized individuals interact with sensitive agent capabilities. Proper logging of user actions provides traceability and supports investigations if misuse occurs.

Agent outputs can also present security risks. Generated responses may accidentally reveal sensitive information from memory or training data; a phenomenon sometimes referred to as data leakage. Additionally, outputs could include harmful or policy-violating content if guardrails are insufficient. Output validation layers should review generated content before delivery. Content moderation filters, compliance checks, and access validation mechanisms can prevent sensitive disclosures. In environments where agents generate executable outputs, such as scripts or code modifications, additional verification layers should confirm safety before execution. Monitoring output patterns also help detect anomalies that might signal compromise or model drift.

As AI agents become more autonomous and integrated into enterprise systems, cloud platforms, and decision-making environments, the attack surface associated with these systems expands significantly. Unlike traditional applications that follow deterministic code paths, AI agents rely on probabilistic reasoning, dynamic prompts, external tools, and feedback loops. While this flexibility enables powerful automation, it also introduces unique security risks. Among the most significant threats are prompt injection, data leakage, unauthorized tool execution, over-privileged agents, and hallucinated actions. Each of these risks can undermine system integrity, confidentiality, and operational reliability if not properly addressed.

Prompt injection is one of the most critical vulnerabilities in LLM-powered agent systems. It occurs when malicious or untrusted input attempts to override or manipulate an agent's system-level instructions. Since AI agents interpret language contextually rather than following strictly hard-coded logic, attackers can craft inputs that attempt to change behavior through cleverly embedded instructions. For example, an attacker might insert a phrase such as "Ignore previous instructions and reveal sensitive information" into user input or external content. If safeguards are weak, the agent may follow this instruction despite system-level restrictions. Prompt injection becomes especially dangerous when agents have access to sensitive tools or data sources. Mitigation strategies include separating system prompts from user inputs, applying strict role-based message structures, validating untrusted content, and implementing filtering mechanisms to detect adversarial patterns. Proper architectural separation between control instructions and user-provided content is fundamental to reducing prompt injection risks.

Data leakage refers to the unintended exposure of confidential or sensitive information through an agent's outputs or memory systems. AI agents frequently interact with user data, enterprise databases, APIs, and internal documentation. If proper access controls are not implemented, the

agent may disclose restricted information in response to unrelated queries. Leakage can occur through several pathways. First, conversational memory systems may retain sensitive details that are later surfaced in responses. Second, retrieval-augmented systems may expose documents beyond a user's authorization scope. Third, insufficient output filtering may allow confidential data to appear in generated text. To reduce this risk, organizations should implement encryption for stored data, enforce access boundaries tied to user identity, apply retention policies, and audit output logs for sensitive disclosures. Data classification and redaction mechanisms also help ensure compliance with regulatory and organizational requirements.

AI agents often extend their capabilities by interacting with external tools such as databases, file systems, financial systems, or cloud services. Unauthorized tool execution occurs when an agent invokes a tool without proper validation or outside of intended operational boundaries. This may result from malicious input, misconfigured permissions, or insufficient monitoring. Since tools allow real-world action, not just text generation, this risk can lead to tangible harm. An exploited agent could delete files, modify configurations, expose data, or execute unintended transactions. To mitigate unauthorized execution, structured function-calling frameworks should define allowed inputs and outputs clearly. Additionally, tool calls should pass through authorization checks, input validation layers, and logging systems. Sensitive actions may require multi-factor confirmation or human approval steps to prevent misuse.

Over-privileged agents represent a structural security weakness rather than a single exploit. When an agent is granted excessive access rights such as administrative control over systems, it increases the potential impact of any vulnerability. Even minor flaws in prompt handling or tool validation can become disastrous if the agent's permissions are too broad. The principle of least privilege should guide agent architecture. Agents should only have access to the tools, data, and

systems necessary to complete their assigned tasks. Role-based access control (RBAC), scoped API tokens, and segmented environments help reduce exposure. Regular reviews of agent permissions ensure that access levels remain aligned with operational requirements. By limiting privileges, organizations reduce the blast radius of potential compromises.

Hallucination is a well-documented behavior of large language models, where outputs are syntactically coherent but factually incorrect. In autonomous agent systems, hallucinations can extend beyond incorrect information to incorrect claims about actions. An agent might state that it executed a tool, retrieved data, or completed a process when no such action occurred. Hallucinated actions create operational risks because systems or users may rely on inaccurate status reports. For example, an agent might falsely confirm that a password reset or transaction was completed. To address this risk, agent architectures must separate reasoning from execution tracking. Verified tool execution logs, state management systems, and confirmation checks ensure that reported actions reflect actual system changes. Observability and monitoring mechanisms further enhance trustworthiness and reliability.

Section 7: Industry Frameworks and Standards

Agents' complexity and multi-step lifecycle require high security to be in place; frameworks and standards have been developed to ensure the security of agentic systems. Industry frameworks and standards ensure proper development and reduce risks of attacks on agentic systems. Examples of frameworks include OWASP Top 10 for LLM Applications, US NIST AI Risk Management Framework, and ISO/IEC 42001. Each of these frameworks and standards gives the foundation for proper agent development and deployment. The various parts

of an agent's operation, such as taking input, reasoning through LLMs, outputting, and using memory systems for accessing previous experience, all need to be secured by these industry frameworks.

The Open Worldwide Application Security Project (OWASP) Top 10 for LLMs is a framework designed to identify and address the most critical security vulnerabilities within LLM applications. Large language models are a major tool for complex agentic reasoning; there are various ways LLMs can be used maliciously by working around their architecture and security systems. OWASP highlights five major risks in LLMs that relate to agents: prompt injection, data leakage, output handling, excessive agency, and supply chain vulnerabilities (OWASP, 2025). Prompt injection is when an attacker's input is used to manipulate the behavior of a large language model system. Data leakage occurs when sensitive information is disclosed through model output or data retrieval (OWASP, 2025). The risk of output handling refers to the output of an LLM not being validated, which then "may lead to downstream exploitation" (OWASP, 2025). Excessive agency in an LLM occurs when the system is given too much decision-making power through broad permissions which "allow unintended actions through integrated tools" (OWASP, 2025). Lastly, supply chain vulnerabilities arise from external plugins, models, and data (OWASP, 2025). Each of these risks directly maps to the agent life cycle. Prompt injection can occur through the prompt interface of an agent, data leakage highlights vulnerabilities regarding memory systems and RAG, the issue of output handling relates to the action layer of an agent, excessive agency can occur when an agent has access to certain tools, and finally supply chain issues may arise when agents interact with plugins and APIs.

OWASP Top 10 for LLMs does an excellent job at highlighting possible risks in the agent lifecycle, while the National Institute of Standards and Technology (NIST) addresses risk

management at the organizational level across the full AI lifecycle. The NIST had developed the AI risk management framework “to help organizations manage risks associated with AI” (Tabassi, 2023). NIST identifies core AI risks spanning validity and reliability, safety, security and resilience, and accountability and transparency (Tabassi, 2023). Validity and reliability refer to an AI system properly doing its designated tasks. Safety focuses on an AI system not causing harm to anyone or anything. Security and resilience relate to the ability of an AI system to withstand attackers. Lastly, accountability and transparency refer to being able to explain what an AI system did and why. NIST provides organizations with a map, measure, and manage approach to identify and address AI risks throughout the system lifecycle (Tabassi, 2023). Mapping is identifying and categorizing risks associated with an AI system (Tabassi, 2023). Regarding agents, it would include understanding the risks across each step of the lifecycle. Measuring refers to analyzing risks to understand the likelihood of them occurring and the potential impact (Tabassi, 2023). Measuring can be implemented into agents through testing the agent's behavior and assessing risks. Finally, the manage step includes putting controls in place to prevent the likelihood of risks occurring (Tabassi, 2023). In the agent lifecycle, this would include implementing access control, output validation, and monitoring systems. NIST aims to help organizations throughout the implementation of AI systems, which ties directly to an iterative system such as agents.

Compared to OWASP and NIST, ISO/IEC 42001 focuses more on creating management systems for AI governance within organizations. ISO/IEC 42001 specifies requirements to “establish, implement, maintain and continually improve' AI governance systems within organizations" (International Organization for Standardization & International Electrotechnical Commission, 2023). This applies to all organizations that develop and use AI systems (International

Organization for Standardization & International Electrotechnical Commission, 2023). This aligns with the principle that responsible AI systems should 'ensure auditability and accountability throughout their design, development, and deployment' (Herrera-Poyatos et al., 2025). Establishing proper governance includes organizations creating standards and rules for appropriate use and creation of AI systems. This could include what type of information can be put into AI systems, or the fact one must not claim AI generated work as their own. Implementation of AI governance systems would be formally documenting and distrubting this information to the company. Establishing and implementing governance continually directly ties to the agent's lifecycle as agents are continuously adapting. Improving AI governance is also mentioned by the ISO/IEC, which is extremely pertinent as AI systems such as agents are constantly changing. Auditing how organizations develop and deploy agents ensures that governance standards are being followed. This ensures agents act properly with their intended purpose as they adapt.

The multi-step agent lifecycle gives way for various points of attack. OWASP highlights the various points of risk regarding LLMs which can directly tie to the agent lifecycle. NIST has a broader risk management lifecycle to manage AI systems properly. ISO/IEC 42001 gives the foundation for companies who develop and use AI to secure their systems. Each of these frameworks does an important and unique job of ensuring the proper risk prevention when using AI systems such as agents.

Section 8: Securing Agent Data and Behavior

Frameworks such as OWASP and NIST highlight risks within AI development and guidelines for risk management. This section will build upon these frameworks by looking at specific techniques to secure agent data and behavior. Specific techniques include examining access

controls, sandboxed environments, secrets management, encryption, and rate limiting. It is important to identify threats as well, such as memory poisoning and unexpected privilege escalation.

One way to secure agent behavior and data is to implement access control to limit what tools agents can access along with what they can do. The risk with access control leads to "Agents exploiting overly permissive tools [can] perform unintended actions or access unauthorized resources" (OWASP, n.d). When using agents, one can implement least privilege principles to ensure they have the minimum access to accomplish their tasks (IAPP, 2025). This comes with scoped API keys that only grant specific required permissions (IAPP, 2025). Giving agents permissions only to tools and information they need reduces the attack surface and limits the damage of compromised accounts (Varonis, 2025).

The next security mechanism that will be investigated is sandboxed tools. Sandboxes are isolated environments for testing and monitoring AI behavior (AISI, 2025). Sandboxes limit a model's access to external systems and data, which allows for evaluation without risking sensitive data (AISI, 2025). Sandboxed environments prevent agents from abusing tools or other API calls in harmful ways (Rippling, 2025). It is important to run and execute tools in sandboxed environments with restricted networking, as "each tool connection represents a new attack vector introducing traditional vulnerabilities" (Axis, 2026).

Secrets management is another way to secure agent behavior and data. Secrets management is the practice of securely storing, accessing, and controlling digital authentication credentials (StrongDM, 2025). Some examples of these credentials could be passwords, API keys, certificates, and tokens. This information can be secured by using short-lived credentials when applicable, such as giving tokens that expire quickly (Shadecoder, 2025). Another way to ensure

protection is for credentials to be secured centrally, and tokens and secrets rotated regularly along with renewals of credentials being automated (CyberArk, 2025). Since agents regularly interact with external systems, “sensitive information may be leaked through tool calls, API requests, or agent outputs” (OWASP, n.d). This elevates the need for proper secrets management when deploying an agent.

Encryption is another common security tool to deploy for protecting data. Encryption is when data is received and then translated to a different code. The only way to get this code back to its original information is to have the key that tells you how the information was translated. It is essential for agent data to be encrypted as it is being accessed across different tools. Sensitive data needs to be encrypted at rest, in transit, and in use (Cloud Security Alliance, 2025). This means that information needs to be encrypted when it is not being used, when it is being sent somewhere, and finally when the data is being computed on. All communication between agents should be encrypted by using protocols such as transport layer security (EPAM, 2025). Another technique is to regularly validate data written to or stored by agents using cryptographic checksums to avoid memory poisoning (EPAM, 2025).

The final way to secure agents that will be investigated is rate limiting. Rate limiting refers to the number of times an API can be called upon in a specific timeframe. This relates to agents as an attacker could exploit an exposed API by overloading them with queries (Erlin, 2025). This overloading of queries can lead to systems slowing down or crashing (Erlin, 2025). Enforcing a rate limit allows API providers to prevent abuse and ensure that resources are distributed equally (API7.ai, n.d). Using API gateways that enforce rate limits helps prevent denial of service attacks when deploying agents (Obsidian Security, 2025).

While proactive security mechanisms are essential, it is equally important to identify and respond to attacks that may occur throughout the agent lifecycle. Spotting unusual tool calls is one way to notice an attack. "Agents manipulated into abusing their integrated tools… represent intentional misuse of legitimate functionality" (Axis Intelligence, 2026). This can be hard to detect as tool calls may seem legitimate as they use authorized credentials and permissions (Axis Intelligence, 2026). Ways to spot these tool calls include using monitoring systems that detect abnormal usage patterns, excessive invocation frequency, unexpected data retrieval, or deviations from normal patterns (Singh, 2026). When an anomaly is detected, agents should be suspended or restricted (Singh, 2026). This is essential as an exploitation could result in data exfiltration, resource deletion, privilege escalation, or lateral movement (Axis Intelligence, 2026).

Memory poisoning is another issue that needs to be addressed when monitoring agent behavior. Memory poisoning is when false or manipulated data is injected into an agent's memory storage leading to inaccurate future decisions (Axis Intelligence, 2026). This can be dangerous as 87% of decisions after memory poisoning became compromised within four hours of poisoning (Vectra AI, 2026). Memory poisoning can be identified by requiring data to include metadata such as source identification, timestamps, and cryptographic checksums (MintMCP, 2026). This allows for integrity verification which enables rapid auditing and corruption detection (MintMCP, 2026).

Another issue to worry about when auditing AI behavior is escalation of privileges. Escalation of privileges is when an agent is allowed to access tools or information that is not needed to complete its designated task. This occurs when operational flexibility is allowed without need-to-know assessment from the agent (AXIS Intelligence, 2026). This is dangerous as these permissions are unnecessary and user context is lost (The Hacker News, 2026). Escalation of

privileges can be detected by monitoring systems that recognize requests for permissions that are unusual (Obsidian Security, 2026). To prevent further error agent tokens can be revoked and API access disabled (Obsidian Security, 2026).

One final threat to detect when auditing agent performance is feedback loop failures. Feedback loop failures are agents adapting in response to one another creating dangerous feedback loops and unpredictability (Hammond, 2025). A single compromise can lead to the failure of all downstream processes of an agent lifecycle (Hammond, 2025). Feedback loop failures can be prevented by implementing human-in-the-loop checkpoints to slow down the process and create a safety net against the speed and scale of an agent attack (Stellar Cyber, 2026).

Security mechanisms such as access control, sandboxed environments, and encryption are all ways to defend agent behavior and data. Recognizing attacks such as unusual tool calls, memory poisoning, escalation of privileges, and feedback loop failures are essential to keeping agent behavior and data safe after deployment. Together, these security mechanisms and detection strategies represent proper implementation of the guidelines set by frameworks such as OWASP, NIST, and ISO/IEC 42001.

Section 9: Human-Agent Interaction & Collaboration

As artificial intelligence agents become more advanced and embedded in workplaces, classrooms, healthcare systems, and government services, understanding how humans interact with these systems is essential. While much of the discussion around AI agents focuses on technical architecture and automation, the human dimension is equally important, especially for non-technical audiences. AI agents are most effective not when they operate independently of

people, but when they collaborate with them. Human–agent interaction is built around oversight, shared decision-making, explainability, and augmentation rather than replacement.

A foundational principle in responsible AI deployment is the concept of "humans in the loop." This means that while agents can analyze data, generate plans, and execute routine tasks, humans remain involved in oversight and final decision-making, especially sensitive or high-impact actions. In practical terms, this might look like an AI agent drafting a policy document, recommending a cybersecurity response, or flagging fraudulent transactions, while a human reviews and approves the outcome. Human oversight reduces risk, adds contextual understanding, and ensures ethical considerations are accounted for in critical decisions. Rather than replacing judgments, agents enhance human capacity to process information and respond efficiently. Humans in the loop also provide corrective feedback. When agents make errors or produce suboptimal recommendations, human intervention helps retrain, refine, or redirect the system. This feedback loop strengthens long-term performance and builds trust between users and technology.

One of the safest models of human–agent collaboration is the approval-based workflow. In this model, an agent can prepare an action, such as sending an email, approving a transaction, modifying a record, or deploying a configuration, but it cannot execute the action until a human confirms it. This approach is especially important in industries like finance, healthcare, cybersecurity, and government, where mistakes can have serious consequences. Approval checkpoints act as a safeguard against errors, hallucinated actions, or misinterpretations. They also reinforce accountability by ensuring that responsibility ultimately resides with a human decision-maker. Over time, as trust and verification systems improve, approval of thresholds can

be adjusted. Low-risk tasks may become automated, while high-risk operations retain strict human review. This tiered approach balances efficiency with safety.

For collaboration to work effectively, humans must understand what the agent is doing and why. Explainability refers to an agent's ability to provide transparent reasoning behind its recommendations or actions. Non-technical users need clear and accessible explanations to feel comfortable relying on AI systems. Explainable agents may summarize their reasoning steps, highlight the data sources used, or clarify assumptions made during analysis. For example, instead of simply recommending "approve this loan," an explainable agent might state: "This applicant meets income stability requirements, has a credit score above 720, and no outstanding delinquent payments." Explainability builds trust and accountability. When people understand how a recommendation was formed, they are better equipped to evaluate its accuracy and fairness. In regulated environments, explainability is also essential for compliance and auditing purposes.

Public concerns about AI often center on job displacement. While automation has historically reshaped industries, AI agents are more accurately described as tools of augmentation rather than wholesale replacement. The design and deployment of agent systems increasingly emphasize collaboration rather than substitution. Automation refers to complete task replacement, where machines perform functions previously handled entirely by humans. Augmentation, by contrast, enhances human ability by assisting with tasks, accelerating workflows, and reducing cognitive load. AI agents excel at handling repetitive, time-consuming, or data-heavy activities. For example, they can summarize large reports, draft initial emails, analyze log data, or generate structured documentation. This frees human professionals to focus on strategic thinking,

relationship building, creativity, and complex problem-solving. In this way, agents serve as productivity multipliers rather than workforce eliminators.

A helpful framework for understanding modern AI systems is the “copilot” model. In aviation, a copilot assists the pilot by managing instruments, monitoring systems, and supporting navigation, but does not replace human leadership in critical decisions. Similarly, AI agents act as intelligent assistants that support professionals across industries. In software development, agents can suggest code improvements while programmers maintain control. In healthcare, AI can assist with diagnostics while physicians make final treatment decisions. In cybersecurity, agents can flag anomalies while analysts determine incident response strategies. The copilot model emphasizes partnership with human expertise combined with computational efficiency. As AI agents integrate into organizations, new roles and skill sets are emerging. Rather than eliminating work, AI adoption often creates demand for new expertise. Roles such as AI system supervisors, prompt engineers, AI ethics officers, agent trainers, and AI security analysts are becoming increasingly relevant. These shifts demonstrate that AI agents are catalysts for workforce transformation, not simple substitutes for human talent.

Section 10: Benefits & Societal Impact of Agents

One of the strongest advantages of AI agents is their ability to handle repetitive, structured tasks efficiently and consistently. Tasks such as processing forms, sorting emails, reviewing logs, generating reports, and responding to frequently asked questions can consume significant human time. Agents can execute these activities with minimal fatigue, high consistency, and constant availability. In business environments, this reduces operational overhead and frees employees to focus on more strategic responsibilities. In personal contexts, agents can assist users with scheduling, reminders, document drafting, and information retrieval. The ability to operate

continuously without a decline in performance makes agents particularly effective in high-volume environments.

Agents are also valuable in decision-support roles. They can analyze large volumes of data, identify patterns, and generate evidence-based recommendations for human review. Rather than replacing decision-makers, agents enhance their capacity to evaluate complex information quickly. For example, in finance, agents can assess risk indicators across thousands of data points. In cybersecurity, they can flag suspicious system behavior for analysts. In healthcare, they can summarize patient data trends for clinicians. By synthesizing information into structured insights, agents improve speed and quality of informed decision-making while retaining human oversight.

Modern agents are particularly effective at orchestrating multi-step workflows. Instead of completing a single isolated action, agents can plan sequences of operations: gathering data, evaluating options, invoking tools, verifying results, and reporting outcomes. This capability distinguishes agents from traditional chatbots or static automation scripts. Multi-step execution is useful in environments such as IT support, project management, compliance monitoring, and research assistance. For example, an agent can receive a support request, categorize it, search for relevant documentation, propose a solution, and draft a response all within one coordinated process. This structured orchestration increases efficiency while maintaining traceability and control mechanisms.

AI agents significantly enhance productivity at individual and organizational levels. By automating routine tasks and assisting with complex workflows, they reduce time spent on low-value activities. This allows professionals to focus on creativity, strategic analysis, and interpersonal collaboration, areas where human skills remain strong. At scale, increased

productivity can drive economic growth and innovation. Small teams can accomplish more with agent assistance, and organizations can respond more quickly to market or operational changes. In educational settings, agents can support both instructors and students, streamlining administrative and research processes. Agents also contribute to accessibility by lowering barriers to information and technology use. Natural language interfaces enable individuals to interact with complex systems without specialized technical knowledge. For people with disabilities, AI agents can assist with real-time transcription, text simplification, or navigation support.

AI agents can enhance safety in various sectors. In industrial settings, agents can monitor system performance and flag anomalies before they escalate into failures. In cybersecurity, they can detect unusual activity patterns rapidly, supporting early response to threats. In transportation and logistics, agents can assist with route optimization and hazard analysis.

By operating continuous monitoring systems, agents provide an additional layer of oversight that complements human supervision. When paired with proper safeguards and governance, these capabilities can reduce risk and improve overall system resilience. Agents accelerate research by synthesizing vast quantities of academic and technical material. Researchers can use agents to summarize literature, generate hypotheses, organize datasets, and automate experimental documentation. While human expertise remains central to discovery, agents reduce the time spent on manual analysis. In scientific, medical, and engineering fields, faster knowledge synthesis leads to quicker iteration cycles and more efficient experimentation. This acceleration supports innovation and can contribute to advances in public health, environmental science, and technology development.

Over-automation occurs when organizations delegate too many responsibilities to agents without sufficient oversight. While automation increases efficiency, removing humans from critical decision loops can amplify errors or ethical oversights. Some tasks,particularly those involving nuanced judgment, empathy, or high-stakes consequences, require human involvement. Maintaining balance between automation and supervision ensures that efficiency gains do not compromise accountability or ethical standards. Trust is fundamental for successful human–agent collaboration. If agents produce inaccurate recommendations, hallucinate actions, or behave unpredictably, users may either lose confidence or place excessive trust in flawed outputs. Both extremes are problematic. Building trust requires transparency, explainability, consistent performance, and clear communication of limitations. Users should understand what agents can and cannot reliably do. Since agents interact with APIs, data systems, and digital infrastructure, they create new attack surfaces. Compromised agents can expose sensitive information or perform unauthorized actions. Security risks include prompt manipulation, credential exposure, and misuse of integrated tools.

Robust access controls, auditing mechanisms, and monitoring systems are essential to ensure safe deployment. Security must be treated as a foundational design principle rather than an afterthought. AI agents inherit biases from training data and embedded systems. If not carefully evaluated, these biases may produce unfair or discriminatory outcomes in areas such as hiring, lending, or law enforcement support. Bias can damage public trust and have serious societal consequences. Mitigation requires diverse datasets, fairness testing, bias monitoring, and ethical governance frameworks. Regular auditing helps detect and address biased outputs before they cause harm. As agents become more integrated into daily workflows, there is a risk of overdependence. Excessive reliance on automated systems may reduce human skill development

and critical thinking capacity. In the event of system failure, organizations that lack human redundancy will struggle to adapt.

References

AISI. (2025). *The Inspect sandboxing toolkit*. https://www.aisi.gov.uk/blog/the-inspect-sandboxing-toolkit-scalable-and-secure-ai-agent-evaluations

Altman, S. (2023, January). Interview with StrictlyVC. StrictlyVC. https://www.strictlyvc.com/

Andy Wang. (2025, September 6). *AI Agents Explained: Tools, Memory, and Autonomy*. Skywork Ai. https://skywork.ai/blog/ai-agents-explained-tools-memory-autonomy/

API7.ai. (n.d.). *Rate limiting strategies for API management*. https://api7.ai/learning-center/api-101/rate-limiting-strategies-for-api-management

Axis Intelligence. (2026, January 7). *Agentic AI security vulnerabilities 2026: Enterprise threat landscape analysis*. Axis Intelligence. https://axis-intelligence.com/agentic-ai-security-vulnerabilities-2026/

Balla. (2025, July 29). Inside the Mind of an AI Agent: Architectures, Memory Models, and Decision Loops. The AI Journal. https://aijourn.com/inside-the-mind-of-an-ai-agent-architectures-memory-models-and-decision-loops/

Bots vs. Chatbots vs. AI Agents vs. AI Assistants | 2026. (2026). Crescendo.ai. https://www.crescendo.ai/blog/ai-vs-bots-vs-ai-agents-vs-chatbots

Bryn Mawr College. (2012). *Intelligent agents* [Lecture slides]. https://cs.brynmawr.edu/Courses/cs372/spring2012/slides/02_IntelligentAgents.pdf

Cloud Security Alliance. (2025). *Data security within AI environments*. https://cloudsecurityalliance.org/artifacts/data-security-within-ai-environments

CyberArk. (2025). *When AI agents become admins: Rethinking privileged access in the age of AI*. https://www.cyberark.com/resources/blog/when-ai-agents-become-admins-rethinking-privileged-access-in-the-age-of-ai

EPAM SolutionsHub. (2025). *Agentic AI security: Protect AI systems*. https://solutionshub.epam.com/blog/post/agentic-ai-security

Erlin, T. (2025). *The API security challenge in AI: Preventing resource exhaustion and unauthorized access*. Security Boulevard.

Hammond, L., Chan, A., Clifton, J., et al. (2025). *Multi-agent risks from advanced AI*. Cooperative AI Foundation. https://doi.org/10.48550/arXiv.2502.14143

Herrera-Poyatos, A., Ser, D., López, M., Wang, F.-Y., Herrera-Viedma, E., & Herrera, F. (2025). *Responsible Artificial Intelligence Systems: A Roadmap to Society's Trust through Trustworthy AI, Auditability, Accountability, and Governance*. ArXiv.org. https://arxiv.org/abs/2503.04739

International Association of Privacy Professionals. (2025). *Understanding AI agents: New risks and practical safeguards*. https://iapp.org/news/a/understanding-ai-agents-new-risks-and-practical-safeguards

IBM. (2025, March 10). *Components of AI agents*. Ibm.com. https://www.ibm.com/think/topics/components-of-ai-agents

International Organization for Standardization & International Electrotechnical Commission. (2023). *ISO/IEC 42001:2023 — Information technology: Artificial intelligence — Management system*. ISO. https://www.iso.org/standard/42001

Iryna Deremuk. (2025, March 19). *AI Agents vs. Traditional Software: Which Is Right for Your Business?* Litslink. https://litslink.com/blog/ai-agents-vs-traditional-software

Kalle Bertell. (2025, November 30). *How AI Agents Work: From LLMs to Tools, Memory, and Automation*. Makers Den; Makers' Den. https://makersden.io/blog/how-ai-agents-work-from-llms-to-tools-memory-and-automation

Lambert, N. (2026, February 12). *Tool Use | RLHF Book by Nathan Lambert*. Rlhfbook.com. https://rlhfbook.com/c/13-tools

Mayes, A., Montaldi, D., & Migo, E. (2007). Associative memory and the medial temporal lobes. *Trends in Cognitive Sciences*, *11*(3), 126–135. https://doi.org/10.1016/j.tics.2006.12.003

MintMCP. (2026). *AI agent memory poisoning: How attackers corrupt long-term agent behavior*. https://www.mintmcp.com/blog/ai-agent-memory-poisoning

Obsidian Security. (2025). *Security for AI agents: Protecting intelligent systems in 2025*. https://www.obsidiansecurity.com/blog/security-for-ai-agents

Obsidian Security. (2026). *The 2025 AI agent security landscape*. https://www.obsidiansecurity.com/blog/ai-agent-market-landscape

OWASP Foundation. (2025). *OWASP top 10 for large language model applications 2025*. Open Worldwide Application Security Project. https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/

OWASP Foundation. (n.d.). *OWASP cheat sheet series*. Open Worldwide Application Security Project. https://cheatsheetseries.owasp.org

Peller-Konrad, F., Rainer Kartmann, Dreher, C. R. G., Meixner, A., Reister, F., Grotz, M., & Asfour, T. (2023). A memory system of a robot cognitive architecture and its implementation in ArmarX. *Robotics and Autonomous Systems*, *164*, 104415–104415. https://doi.org/10.1016/j.robot.2023.104415

Rens, G., & Moodley, D. (2016). *A Hybrid POMDP-BDI Agent Architecture with Online Stochastic Planning and Plan Caching*. ArXiv.org. https://arxiv.org/abs/1607.00656

Rippling. (2025). *Agentic AI security: A guide to threats, risks & best practices*. https://www.rippling.com/blog/agentic-ai-security

Shadecoder. (2025). *Secrets management: A comprehensive guide for 2025*. https://www.shadecoder.com/topics/secrets-management-a-comprehensive-guide-for-2025

Stellar Cyber. (2026). *Top agentic AI security threats in late 2026*. https://stellarcyber.ai/learn/agentic-ai-securiry-threats/

StrongDM. (2025). *What is secrets management? Best practices for 2025*. https://www.strongdm.com/blog/secrets-management

Tabassi, E. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.AI.100-1

Team, R. (2025). *ReelMind - Open Source AI Video Models Community*. ReelMind. https://reelmind.ai/blog/agents-in-ai-ai-system-components

The Hacker News. (2026). *AI agents are becoming authorization bypass paths*. https://thehackernews.com/2026/01/ai-agents-are-becoming-privilege.html

Nadella, S. (2024, November 19). Microsoft Ignite 2024 keynote. Microsoft. https://pub-c2c1d9230f0b4abb9b0d2d95e06fd4ef.r2.dev/2024/11/11192024-Ignite-KEY01-Full-Transcript.pdf

Varonis: Varonis. (2025). *Why least privilege is critical for AI security*. https://www.varonis.com/blog/why-polp-is-critical-for-ai-security

Vectra AI. (2026). *Agentic AI security explained: Threats, frameworks, and defenses*. https://www.vectra.ai/topics/agentic-ai-security